

Strategic Ontology Engineering in the Era of Generative AI: Frameworks, Applications, and Measurement Strategies

The landscape of knowledge representation and semantic web technologies has undergone a profound transformation through the integration of large language models (LLMs) and advanced natural language processing architectures. Historically, ontology construction was a highly labor-intensive, manual endeavor requiring intense collaboration between domain experts and knowledge engineers. Today, the synthesis of symbolic logic with parametric neural models—often referred to as neuro-symbolic AI—has opened new frontiers in automated and semi-automated ontology learning. This evolution is not merely an academic exercise; it forms the foundational infrastructure for enterprise-grade artificial intelligence (AI), powering architectures like Graph-based Retrieval-Augmented Generation (GraphRAG) and enabling strict regulatory compliance under frameworks like the European Union Artificial Intelligence Act (EU AI Act).

This comprehensive report examines the real-world applications and methodological approaches for constructing ontologies based on unstructured text and use-case definitions. It subsequently provides an exhaustive analysis of the evaluation metrics, validation tools, and deployment strategies required to measure and maintain the efficacy of these semantic architectures in dynamic production environments.

Methodological Frameworks for Ontology Construction

The transition from the "art" of building single, isolated ontologies to a rigorous engineering discipline has been formalized by several mature frameworks. While early paradigms like METHONTOLOGY and On-To-Knowledge established the foundational knowledge-level construction lifecycle, modern industrial applications require frameworks that support distributed collaboration, dynamic evolution, and rapid integration into software development pipelines¹.

The NeOn Framework and Scenario-Based Development

The NeOn Methodology represents a paradigm shift by aiming at building ontology networks rather than isolated models. It relies heavily on a scenario-based architecture designed to support the reuse of existing resources, collaborative development, and dynamic evolution in distributed environments¹. The framework defines nine specific scenarios for developing ontology networks, offering unprecedented flexibility depending on the starting conditions of an enterprise's data landscape.

NeOn Scenario	Description and Use Case
Scenario 1	From specification to implementation: The ontology network is developed from scratch (without reusing existing resources). Requires detailed requirements specification and scheduling ³ .
Scenario 2	Reusing and re-engineering non-ontological resources (NORs): Domain experts decide which NORs (e.g., text corpora, databases) can be reused and transformed into ontologies ³ .
Scenario 3	Reusing ontological resources: Developers use existing ontologies, ontology modules, or individual statements as building blocks for the network ³ .
Scenario 4	Reusing and re-engineering ontological resources: Existing models are not only imported but structurally adapted to new use cases ³ .
Scenario 5	Reusing and merging ontological resources: Several ontological resources in the same domain are selected and merged into a new, consolidated resource ³ .
Scenario 6	Reusing, merging, and re-engineering: An extension of Scenario 5, where the merged resources undergo additional structural transformation ³ .
Scenario 7	Reusing Ontology Design Patterns (ODPs): Developers access repositories to leverage proven design patterns for modeling complex situations ³ .
Scenario 8	Restructuring ontological resources: Existing models are modularized, pruned, extended, or specialized to be optimally

	integrated into the new network ³ .
Scenario 9	Localizing ontological resources: The ontology is adapted to other languages and cultures, creating multilingual knowledge representation ³ .

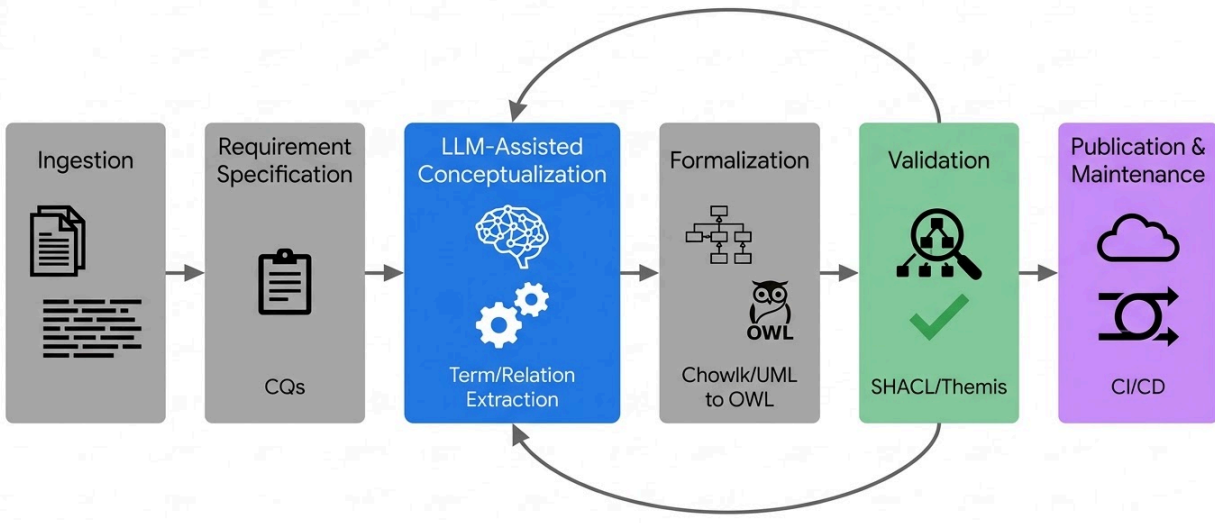
The defining characteristic of NeOn is its emphasis on integration and reuse, explicitly addressing the limitations of prior methods that assumed ontologies were built in a vacuum. By providing precise guidelines for requirement specification, scheduling, and resource localization, NeOn transforms ontology creation into a predictable, measurable software engineering process¹. This "divide and conquer" approach decomposes the general problem of network development into solvable subproblems, facilitating rapid assimilation by software teams¹.

The LOT (Linked Open Terms) Methodology and Agile Industrial Alignment

While NeOn offers a comprehensive theoretical and scenario-based approach, the "Linked Open Terms" (LOT) methodology serves as a lightweight, agile alternative optimized for modern software industry standards. LOT adapts Continuous Integration and Continuous Deployment (CI/CD) philosophies to ontology engineering, organizing the development process into four iterative sprints: ontology requirements specification, implementation, publication, and maintenance⁵.

A critical feature of LOT is its intense focus on Competency Questions (CQs) during the requirements specification phase. CQs define the functional scope of the ontology by articulating the exact natural language queries the final system must be able to answer⁵. During implementation, domain knowledge is iteratively organized into conceptual models. Here, visual modeling tools prove highly effective. The "Chowlk" system, for instance, allows domain experts to sketch ontology conceptualizations using Unified Modeling Language (UML) diagrams, which are then automatically translated into the Web Ontology Language (OWL)⁵. This visual approach bridges the semantic gap between domain experts defining the use cases and the technical systems processing the logical axioms⁹.

The industrial alignment of LOT ensures that ontologies are not just academic artifacts but functional components of a software stack. The publication phase mandates that the ontology is accessible via standard web protocols (content negotiation) both as human-readable documentation (e.g., HTML) and machine-readable code (e.g., RDF/XML, Turtle) via its namespace URI, ensuring enterprise-wide interoperability⁵. Maintenance occurs through continuous iterations where domain experts can propose new requirements or improvements via issue tracking systems (like GitHub Actions), integrating seamlessly into CI/CD pipelines⁵.



Large Language Models (LLMs) in Ontology Learning

The application of large language models to Ontology Learning (OL) has fundamentally disrupted traditional engineering timelines. OL involves the automatic or semi-automatic extraction of terms, taxonomic hierarchies, and non-taxonomic relationships from unstructured text data to formulate a formal ontology¹³. Because LLMs implicitly encode massive amounts of relational data within their parametric weights, they possess the latent capability to deduce semantic structures from unstructured corpora¹³.

The LLMs4OL Paradigm and Empirical Benchmarks

The LLMs4OL (Large Language Models for Ontology Learning) initiative established a comprehensive task paradigm to evaluate how effectively foundation models can perform OL tasks in a zero-shot or few-shot setting¹³. The paradigm subdivides ontology learning into discrete subtasks:

Subtask in the LLMs4OL Paradigm	Analytical Focus
Term Extraction & Typing (Term Typing)	Identifying core domain concepts from raw text and categorizing them into broader, semantic types ¹³ .
Taxonomy Discovery	Inducing hierarchical relationships (is-a / subclass-of) to build the structural backbone of the ontology.

Extraction of Non-Taxonomic Relations	Identifying complex, domain-specific predicates (e.g., <i>causes</i> , <i>consists of</i>) that link discrete concepts.
--	--

Empirical evaluations across the LLMs4OL framework reveal stark differences in model capabilities. Highly parameterized proprietary models, such as Claude 3 Sonnet and GPT-4, consistently demonstrate robust zero-shot reasoning capabilities, achieving impressive F1 scores in taxonomy induction and relation extraction¹⁸. Specifically, Claude 3 Sonnet reached an F1 score of 0.967 in certain evaluations, while the Mixtral-8x7B model recorded a strong F1 score of 0.847¹⁸. In contrast, smaller open-source models often require intensive instruction tuning, Chain-of-Thought (CoT) prompting, or Retrieval-Augmented Generation (RAG) techniques to remain competitive¹⁷. An analysis of various LLMs in the LLMs4OL 2025 Challenge (Task A) showed that Qwen2.5-72B-Instruct and Llama-3.3-70B exhibit strong recall capabilities in certain domains (e.g., Scholarly, Engineering), but precision depends heavily on domain complexity²⁰.

However, purely LLM-driven OL is not without severe limitations. LLMs frequently struggle with complex, multi-step logical deductions and are prone to hallucinations—generating structurally plausible but factually incorrect triples²¹. Another critical issue is the "quadratic complexity" of relation prediction, making it computationally unscalable to generate massive ontologies solely through zero-shot edge prediction¹⁴. Furthermore, benchmarks are often criticized for potential data contamination in the training data, which can artificially inflate recall and is inadequately assessed by simple string-matching evaluations¹⁴.

Hybrid Pipelines and NeOn-GPT

To mitigate the unreliability of autonomous LLMs, research has branched into two primary strategies: end-to-end model fine-tuning and hybrid procedural pipelines.

End-to-end approaches, like the OLLM architecture, attempt to fine-tune LLMs to generate entire ontological sub-components in a single pass, sum the generated graphs, and apply post-processing. While effective on constrained datasets (like Wikipedia categories), this approach often fails to seamlessly integrate with enterprise requirements and complex competency questions.

A more pragmatic and industrially viable approach is the hybrid pipeline, prominently exemplified by "NeOn-GPT". NeOn-GPT fuses the rigorous, procedural scaffolding of the NeOn methodology with the generative power of LLMs, creating an "expert-in-the-loop" co-pilot system⁴. Rather than blindly tasking an LLM with generating a complete ontology, NeOn-GPT orchestrates a sequence of highly constrained prompts aligned with the NeOn lifecycle. The LLM is sequentially tasked with scoping the domain, identifying existing ontologies for reuse, generating class hierarchies, creating properties, and populating instances²⁶.

Crucially, the outputs at each stage of the NeOn-GPT pipeline undergo automated syntax validation, consistency checks, and error resolution using external deterministic tools before proceeding to the next step²⁶. This neuro-symbolic orchestration ensures that the LLM's generative flexibility is strictly bounded by formal logical constraints. In case studies, such as the automated modeling of the Stanford Wine Ontology, NeOn-GPT has been shown to

produce logically sound and consistent models, even though the LLM itself lacks deep deductive capabilities⁴. This approach drastically reduces development time (often by 60-80%) while preserving the structural integrity required for enterprise deployment²⁴.

Methodological Trade-offs in Ontology Engineering

Methodology	Precision	Recall	Logical Consistency	Cost & Effort	Scalability
Manual Use-Case Driven (LOT/NeOn)	High (95-99%)	Moderate (Limited by scope)	Deterministic & Reasoner Validated	High Labor Cost & Slow (Months)	Poor (Human Bottleneck)
Fully Automated NLP/LLM (Zero-Shot)	Moderate to Low (12-65%)	High (50-85%)	Frequent Hallucinations & Violations	Low Cost & Near-Instant	High (Massive Corpora Parsing)
Hybrid Human-in-the-Loop (HITL / NeOn-GPT)	High (88-95%)	High (80-92%)	Automated SHACL/OWL Reasoner Validated	Optimal (60-80% Time Reduction)	High (Guided by LLM Co-pilots)

Hybrid methodologies, such as NeOn-GPT, provide the optimal balance, leveraging the scalability of LLMs while relying on deterministic reasoners and human oversight to guarantee logical consistency.

Data source: Research Synthesis

Real-World Enterprise Applications

The pivot toward semantic technologies is driven by high-stakes enterprise requirements where data accuracy, interoperability, and explainability are non-negotiable. Two primary application domains illustrate the necessity of robust ontology engineering: Semantic Layers for AI (like GraphRAG) and Regulatory Compliance.

The Semantic Layer and GraphRAG

In modern enterprise AI architectures, standard RAG (Retrieval-Augmented Generation)—which relies on vector similarity searches over fragmented text chunks—has exposed severe limitations. Standard RAG struggles with multi-hop reasoning (e.g., "A relates to B, which triggers C"), synthesis across multiple documents, and entity-centric queries, generally hitting an accuracy ceiling of around 80%³⁰. Because vector databases treat text chunks as isolated units without understanding the relationships between them, the LLM often receives incomplete or contextually inaccurate information, inevitably leading to hallucinations³¹.

To resolve this, enterprises are transitioning to Graph-based RAG (GraphRAG). GraphRAG anchors information retrieval and reasoning in a deterministic semantic layer consisting of a formal ontology and an instantiated knowledge graph³⁰.

- **The Ontology (The Rules Engine):** Defines the exact meaning of entities, their attributes, and permissible relationships, acting as a strict semantic contract. It disambiguates business logic (e.g., ensuring a "prospect" in sales maps correctly to a "lead" in marketing) and enforces constraints (e.g., "waterproof" is a valid feature for a jacket, but not for a service)³⁰.
- **The Knowledge Graph (The Source of Truth):** Stores the actual operational facts mapped against the ontology. It knows true inventory, pricing, and configurations, regardless of what a language model saw in its training data³³.
- **The Semantic Orchestration Layer:** Translates natural language intent into structured graph queries (like SPARQL or Cypher), ensuring the LLM is provided with explicit, traversable reasoning paths¹⁵.

By explicitly modeling relationships, GraphRAG drastically reduces noise in the system. Industry benchmarks show that systems using well-modeled semantic layers can achieve query accuracy rates between 90% and 99% on complex enterprise tasks, significantly outperforming naive RAG and pure text-to-SQL approaches³⁰. The structured semantics allow answers to be built on explicit graph paths rather than relying on probabilistic estimations³¹.

Regulatory Compliance and the EU AI Act

The enactment of the European Union Artificial Intelligence Act (EU AI Act) has imposed strict legal requirements on high-risk AI systems, demanding unprecedented transparency, data governance, and algorithmic auditability³⁶. Standard machine learning models, particularly deep neural networks, act as "black boxes" and lack the epidemiological, financial, or legal context required for domain experts to validate their outputs³⁶.

Ontologies bridge this "Semantic Gap" by providing a machine-readable, logically grounded audit trail. Through neuro-symbolic frameworks, statistical associations derived from machine learning can be mapped directly to semantic concepts. Under Article 10 of the EU AI Act, for instance, organizations must maintain rigorous data governance. Researchers have successfully mapped these legal requirements to technical standards (like ISO/IEC 5259) using the SKOS (Simple Knowledge Organization System) ontology³⁹. This approach meticulously charts how specific concepts interlock between legal and technical texts, identifying areas of full and partial alignment⁴⁰.

This enables executable compliance modeling. By formalizing regulatory obligations as machine-interpretable entities, compliance officers can use semantic web reasoning (e.g., SPARQL queries over the knowledge graph) to instantly validate whether an AI system's data provenance, bias monitoring, and human oversight mechanisms satisfy specific legal clauses³⁸. The ontology serves as a transparent layer of *Semantic Fidelity*, proving that an AI decision aligns with formalized domain knowledge and withstands axiomatic scrutiny³⁶.

Evaluation Metrics and Measurement Strategies

The utility of an ontology depends entirely on its structural soundness and functional alignment

with the intended use case. Measuring the ontology engineering process requires a multi-dimensional approach that evaluates both the quantitative outputs of LLM assistants and the qualitative, logical integrity of the resulting models.

Quantitative Metrics and the OntoAxiom Benchmark

When measuring the efficacy of LLM-driven ontology learning, standard machine learning metrics—Precision, Recall, and F1-Score—are strictly applied against established benchmarks like *OntoAxiom*⁴².

The OntoAxiom benchmark is specifically designed to test the identification of fundamental ontology components (axioms), such as subclasses, disjointness, domains, and ranges. It encompasses nine medium-sized ontologies with a total of 17,118 triples and 2,771 axioms⁴².

- **Precision:** Measures the exactness of the generated axioms, ensuring the LLM does not hallucinate non-existent or nonsensical concepts⁴³.
- **Recall:** Measures completeness, verifying that the model successfully identified all relevant entities and axioms present in the source text²³.
- **F1-Score:** The harmonic mean, balancing the trade-off between exactness and completeness⁴³.

Recent evaluations using the OntoAxiom benchmark indicate that prompting strategies significantly impact these metrics. An "Axiom-by-Axiom" (AbA) prompting approach—where the LLM is sequentially queried for discrete axioms—yields significantly higher F1 scores than attempting to generate the entire ontology graph in a single, direct prompt⁴⁶. Furthermore, performance varies widely by domain: for subclass axioms, the well-known FOAF ontology achieves a score of 0.642, while the music ontology scores only 0.218⁴⁶.

Evaluating semantic representations also requires advanced clustering metrics like *Semantic Internal Cohesion*. This metric calculates the average pairwise cosine similarity of concept embeddings within a generated cluster, providing a mathematical representation of how closely related and unambiguous a machine-generated conceptual category is¹⁹.

Qualitative Evaluation of Structural Integrity

Beyond pure ML metrics, the ontology itself must be evaluated for semantic richness and graph integrity. Structural evaluations analyze graph metadata, calculating metrics such as:

- **Attribute Richness:** The average number of attributes defined per class.
- **Class/Relation Ratio (CRR):** Assesses whether the ontology is overly taxonomic (concept-heavy) or sufficiently relational⁴⁶.
- **Equivalence Ratio:** Measures the degree of semantic integration and disambiguation across different data sources.

Criteria-based assessments, heavily used in industrial contexts, evaluate the ontology across five core dimensions:

Criterion	Significance for Ontology Quality
Completeness	Ensures that all knowledge required for a

	specific task is modeled at the schema (TBox) and instance (ABox) levels ²² .
Conciseness	Guarantees the absence of redundancies, ensuring no two equivalent attributes exist with different names ²² .
Consistency	Verifies that the ontology is free of contradictory information and logical errors ²² .
Correctness	Assesses whether the ontology truthfully and meaningfully captures the real-world aspects of the domain ⁴⁷ .
Clarity	Measures how unambiguously and clearly terms are described through formal definitions ⁴⁷ .

Automated Validation Tools

To operationalize these qualitative metrics in practice, knowledge engineers rely on robust automated diagnostic tools.

Pitfall Detection with OOPS! (OntOlogy Pitfall Scanner!)

The **OntOlogy Pitfall Scanner! (OOPS!)** is an indispensable, platform-independent tool for detecting structural, functional, and usability-profiling anomalies within an ontology⁴⁸. By analyzing the RDF/OWL code, OOPS! cross-references the model against a catalog of over 40 established modeling pitfalls⁴⁶.

These errors range from minor usability issues (e.g., missing label annotations) to critical logical flaws. A typical critical issue, for example, is P03 (using "is" instead of "rdfs:subClassOf") or the creation of cyclic class hierarchies (Class A is a subclass of Class B, while Class B is simultaneously a subclass of Class A)⁵². Similarly, the tool uncovers the frequent misuse of ontology restrictions like `allValuesFrom` and `someValuesFrom`⁴⁹. OOPS! categorizes these pitfalls by importance (Critical, Important, Minor), allowing developers to efficiently prioritize fixes during engineering sprints.

Functional Validation with Themis

While OOPS! checks *how* the ontology is built, the **Themis** framework verifies *what* it can do⁵⁴. Themis validates the ontology against the original Competency Questions (CQs) defined in the requirements phase. It achieves this by translating natural language CQs into formalized test expressions using lexico-syntactic patterns (LSPs)⁵⁵. By executing these test queries against the ontology, Themis verifies whether the required knowledge has been correctly modeled and whether the underlying reasoner can successfully infer the correct answers. This effectively

provides a Behavior-Driven Development (BDD) testing environment for semantic web applications⁵⁵.

Semantic Validation and CI/CD: The SHACL vs. OWL Paradigm

When ontologies transition from the development environment to active production settings, validation mechanisms must shift to serve complex enterprise data integration (ETL) pipelines. A critical architectural decision in modern ontology deployment is the application of validation languages, primarily the distinction between the Web Ontology Language (OWL) and the Shapes Constraint Language (SHACL)⁵⁷.

Open World vs. Closed World Assumptions

OWL is built on the Open World Assumption (OWA). Under OWA, the absence of a fact does not imply the fact is false; it merely means the fact is currently unknown⁵⁷. While highly advantageous for representing distributed, incomplete knowledge on the World Wide Web, this poses a massive challenge for enterprise data validation. When an OWL reasoner evaluates instance data that violates a defined domain constraint, it does not necessarily throw a validation error. Instead, it might simply infer new facts to resolve the discrepancy, rendering OWL ineffective as a strict quality gatekeeper for data.

To enforce strict data governance, enterprises deploy SHACL, which operates under the Closed World Assumption (CWA). SHACL allows engineers to define explicit "shapes" or constraints that instance data must adhere to⁵⁷. If a knowledge graph ingests data missing a required attribute or violating a datatype restriction, the SHACL processor immediately flags the violation and generates an RDF-based validation report. This enables banks and government agencies to automatically and reliably reject instance data if it does not conform to the predefined structural standard⁵⁷.

Preventing Ontology Drift in CI/CD Pipelines

In production systems, ontologies are not static; they undergo continuous evolution to adapt to shifting business logic or regulatory environments. This ongoing updating introduces the risk of "Ontology Drift"—where the conceptual schema gradually diverges from the underlying data reality, leading to downstream application failures in GraphRAG systems or AI agents³⁵.

To combat ontology drift, enterprises embed SHACL validators and OWL reasoners directly into modern Continuous Integration/Continuous Deployment (CI/CD) pipelines, leveraging infrastructure like GitHub Actions¹². In these automated workflows, every proposed commit to the ontology repository triggers a series of automated tests:

1. **Syntax Check:** Ensures the serialized RDF/Turtle code is valid.
2. **Structural Scan:** Executes OOPS! via API to immediately detect newly introduced modeling pitfalls⁶⁴.
3. **Functional Testing:** Runs Themis test suites to verify that existing Competency Questions continue to be met.
4. **Data Validation:** Uses SHACL processors (e.g., within tools like RDFSX or the PoolParty Semantic Suite) to validate synthetic or production-like knowledge graphs against the

new schema, ensuring backward compatibility and preventing the ingestion of faulty data. In platforms like PoolParty, OWL definitions can be automatically transformed into SHACL shapes via in-store Datalog engines, facilitating the transition from the "Open World" to the "Closed World" of constraint validation without manual intervention.

This rigorous, automated testing infrastructure guarantees that the semantic layer remains a deterministic, highly regulated source of truth, fully capable of grounding the next generation of enterprise AI applications.

Conclusion

The construction and measurement of ontologies have evolved from esoteric semantic web exercises into the critical path for deploying reliable, scalable artificial intelligence in enterprises. Methodological frameworks like NeOn and LOT provide the necessary procedural rigor to structure domain knowledge. Simultaneously, modern hybrid pipelines like NeOn-GPT prove that large language models (LLMs) can rapidly accelerate this process, provided they are guided into orderly channels through human oversight and deterministic logic.

As enterprises increasingly rely on GraphRAG to overcome the accuracy limitations of vector-based retrieval, and as regulatory frameworks like the EU AI Act mandate absolute algorithmic transparency, the role of the ontology is paramount. It acts as an immutable rules engine that imparts meaning, enforces governance, and ensures explainability. By employing robust evaluation metrics—backed by benchmarks like OntoAxiom—utilizing automated validation tools like OOPS! and Themis, and embedding SHACL-based constraints into continuous integration pipelines, organizations can maintain dynamic and highly precise semantic layers. Ultimately, this neuro-symbolic approach, bridging the vast parametric capabilities of language models with the unyielding precision of structured ontologies, represents the most effective path for anchoring trustworthy, context-aware AI systems in reality.

Works cited

1. NeOn Methodology for Ontology Networks | PDF - Scribd, <https://www.scribd.com/document/736909153/71542225>
2. LOT: An industrial oriented ontology engineering framework, https://www.researchgate.net/publication/358985910_LOT_An_industrial_oriented_ontology_engineering_framework
3. The NeOn Methodology - Ontology Engineering Group, <https://oeg.fi.upm.es/index.php/en/methodologies/59-neon-methodology/index.html>
4. NeOn-GPT: Ontology Learning with LLMs | PDF - Scribd, <https://www.scribd.com/document/734849428/ESWC-2024-paper-290>
5. LOT methodology, <https://lot.linkedata.es/>
6. Procedure Model for Building Knowledge Graphs for Industry ... - arXiv, <https://arxiv.org/html/2409.13425v1>
7. Software for delivering intelligence at the edge intermediate release, https://aeros-project.eu/wp-content/uploads/2024/04/aerOS_D4.2_Software-for-

- [delivering-intelligence-at-the-edge-intermediate-release-v1.0-submission.pdf](#)
8. Special Track LLMs for KE - ESWC 2024,
<https://2024.eswc-conferences.org/special-track-llms-for-ke/>
 9. Converting UML based conceptualization into OWL ontologies,
<https://interoperable-europe.ec.europa.eu/collection/oeg-upm/news/converting-uml-based-conceptualization-owl-ontologies>
 10. DTAG: A Methodology for Aggregating Digital Twins Using ... - MDPI,
<https://www.mdpi.com/2076-3417/14/13/5960>
 11. D4.6 – Ontology ecosystem reference implementation - Zenodo,
<https://zenodo.org/records/10964622/files/D4.6%20v1.0.1.pdf?download=1>
 12. D3.3 Hospital Knowledge Base and ODIN semantic ontology v2,
https://odin-smarthospitals.eu/wp-content/uploads/2025/04/D3.3-Hospital-Knowledge-Base-and-ODIN-semantic-ontology-v2_v1.0.pdf
 13. (PDF) LLMs4OL: Large Language Models for Ontology Learning,
https://www.researchgate.net/publication/375197011_LLMs4OL_Large_Language_Models_for_Ontology_Learning
 14. End-to-End Ontology Learning with Large Language Models - arXiv,
<https://arxiv.org/pdf/2410.23584>
 15. LLMs4OL: Large Language Models for Ontology Learning,
<https://dokumen.pub/llms4ol-large-language-models-for-ontology-learning.html>
 16. OntoLearner: A Modular Python Library for Ontology Learning with,
<https://arxiv.org/html/2607.01977v1>
 17. arXiv:2508.19428v1 [cs.CL] 26 Aug 2025, <https://arxiv.org/pdf/2508.19428>
 18. Large Language Models for Scholarly Ontology Generation - arXiv,
<https://arxiv.org/html/2412.08258v2>
 19. LLM-Driven Unsupervised Discovery of Semantic Decision ... - MDPI,
<https://www.mdpi.com/2227-7390/14/15/2681>
 20. (PDF) Phoenixes at LLMs4OL 2025 Task A: Ontology Learning With,
https://www.researchgate.net/publication/396083980_Phoenixes_at_LLMs4OL_2025_Task_A_Ontology_Learning_With_Large_Language_Models_Reasoning
 21. Pipeline for Ontology Construction Using a Large Language Model,
<https://www.scitepress.org/Papers/2025/130960/130960.pdf>
 22. Ontology Generation using Large Language Models - arXiv,
<https://arxiv.org/html/2503.05388>
 23. Automating the Formulation of Competency Questions in Ontology,
https://livrepository.liverpool.ac.uk/3190379/1/201315064_Feb2025.pdf
 24. LLM-supported collaborative ontology design for data and ... - Frontiers,
<https://www.frontiersin.org/journals/big-data/articles/10.3389/fdata.2025.1676477/full>
 25. A Large Language Model-Powered Pipeline for Ontology Learning,
https://www.researchgate.net/publication/382866036_NeOn-GPT_A_Large_Language_Model-Powered_Pipeline_for_Ontology_Learning
 26. A Large Language Model-Powered Pipeline for Ontology Learning,
https://www.researchgate.net/publication/388435283_NeOn-GPT_A_Large_Language_Model-Powered_Pipeline_for_Ontology_Learning

27. Large Language Models for Ontology Learning in Life Sciences - arXiv, <https://arxiv.org/html/2412.02035v1>
28. The Semantic Web: ESWC 2024 Satellite Events, <https://www.springerprofessional.de/en/the-semantic-web-eswc-2024-satellite-events/50569048>
29. [unknown url](#)
30. How to Build a Semantic Layer for Enterprise AI with Fluree, <https://flur.ee/blog/how-to-build-a-semantic-layer-for-enterprise-ai>
31. GraphRAG in the Enterprise, <https://enterprise-knowledge.com/graphrag-in-the-enterprise/>
32. GraphRAG in 2026: A Practical Buyer's Guide to Knowledge-Graph, <https://medium.com/@tongbing00/graphrag-in-2026-a-practical-buyers-guide-to-knowledge-graph-augmented-rag-43e5e72d522d>
33. Ontologies, Knowledge Graphs, and a Semantic Layer - Medium, <https://vimal-dwarampudi.medium.com/your-ai-is-guessing-heres-how-to-make-it-stop-e1874bfa312a>
34. Knowledge Graph Construction for AI: Enterprise Data Graph (2026), <https://atlan.com/know/ai-agent/knowledge-graph/knowledge-graph-construction-for-ai/>
35. Semantic Layer vs. Knowledge Graph: Key Differences (2026), <https://colrows.com/blogs/semantic-layer-vs-knowledge-graph/>
36. A Semantic-Based Explainable Artificial Intelligence Framework for, <https://www.preprints.org/manuscript/202608.0682>
37. AI Cards: Towards an Applied Framework for Machine-Readable AI, <https://arxiv.org/html/2406.18211v1>
38. Automated regulatory compliance for AI systems in the security domain, <https://open-research-europe.ec.europa.eu/articles/6-83>
39. Semantic Mapping of EU AI Act and ISO/IEC 5259 - Trustnoww, <https://www.trustnoww.com/blog/semantic-mapping-eu-ai-act-iso-iec-5259>
40. Mapping Data Governance Requirements Between the European, <https://openreview.net/pdf?id=5ilvDyTkbq>
41. Ontology Services for AI Deployment - Kaiasm, <https://www.kaiasm.com/products/ontology-for-ai/>
42. Ontology Learning with LLMs: A Benchmark Study on Axiom ... - arXiv, <https://arxiv.org/html/2512.05594v1>
43. Exploring large language models for ontology learning, https://iacis.org/iis/2024/4_iis_2024_299-310.pdf
44. Ontology Learning with LLMs: A Benchmark Study on Axiom ... - arXiv, <https://arxiv.org/pdf/2512.05594>
45. ARTIFICIAL INTELLIGENCE FOR MARKET RELATIONSHIP, <https://upcommons.upc.edu/bitstreams/c297dd66-4c1b-44ef-baca-31e08367ecf6/download>
46. Delta: A Modular Ontology Evaluation System - MDPI, <https://www.mdpi.com/2078-2489/12/8/301>
47. Semantic assessment of smart healthcare ontology - Emerald Insight,

- <https://www.emerald.com/ijwis/article-split/16/4/475/165264/Semantic-assessment-of-smart-healthcare-ontology>
48. Validating Ontologies with OOPS! | Request PDF - ResearchGate, https://www.researchgate.net/publication/262368940_Validating_ontologies_with_OOPS
 49. Evaluating Coverage and Correctness using a Domain Corpus, https://www.researchgate.net/publication/370413706_Automated_Ontology_Evaluation_Evaluating_Coverage_and_Correctness_using_a_Domain_Corpus
 50. Methodological Exploration of Ontology Generation with a ... - MDPI, <https://www.mdpi.com/2079-9292/14/14/2863>
 51. Methodology for ontology design and construction - SciELO México, https://www.scielo.org.mx/scielo.php?script=sci_arttext&pid=S0186-10422019000500015
 52. OOPS! (Ontology Pitfall Scanner!) - SciSpace, <https://scispace.com/pdf/oops-ontology-pitfall-scanner-an-on-line-tool-for-ontology-l4irgaxzi8.pdf>
 53. The landscape of ontologies in materials science and engineering, <https://arxiv.org/html/2408.06034v1>
 54. Ontology development basic tools | PDF - Slideshare, <https://www.slideshare.net/slideshow/ontology-development-basic-tools/251096744>
 55. Themis: a tool for validating ontologies through requirements, https://www.researchgate.net/publication/335152619_Themis_a_tool_for_validating_ontologies_through_requirements
 56. Investigating Open Source LLMs to Retrofit Competency Questions, <https://ojs.aaai.org/index.php/AAAI-SS/article/download/31793/33960/35862>
 57. Repairing Inconsistencies in Data Processing for Enterprise, <https://www.oxfordsemantic.tech/blog/repairing-inconsistencies-in-data-processing-for-enterprise-knowledge-graphs>
 58. Knowledge Graphs, Metadata Practices, and Badiou's Mathematical, <https://kula.uvic.ca/index.php/kula/article/view/192/462>
 59. RDFMutate: Mutation-Based Generation of Knowledge Graphs, <https://ebjohansen.org/publication/25-iswc/25-iswc.pdf>
 60. What is OWL (Web Ontology Language)? | Semantic Partners, <https://www.semanticpartners.com/learn/what-is-owl>
 61. Computation and Language - arXiv, <https://arxiv.org/list/cs.CL/new?ref=genaisecretsauce.com>
 62. Ecosystem integration: the use of ontologies in integrating ... - Frontiers, <https://www.frontiersin.org/journals/manufacturing-technology/articles/10.3389/fmtec.2024.1331197/full>
 63. IT roles - ITech.Work, <http://itech.work/jobs>
 64. OOPS! (Ontology Pitfall Scanner!) - Semantic Web Journal, <https://www.semantic-web-journal.net/content/oops-ontology-pitfall-scanner-supporting-ontology-evaluation-line>